December, 2025



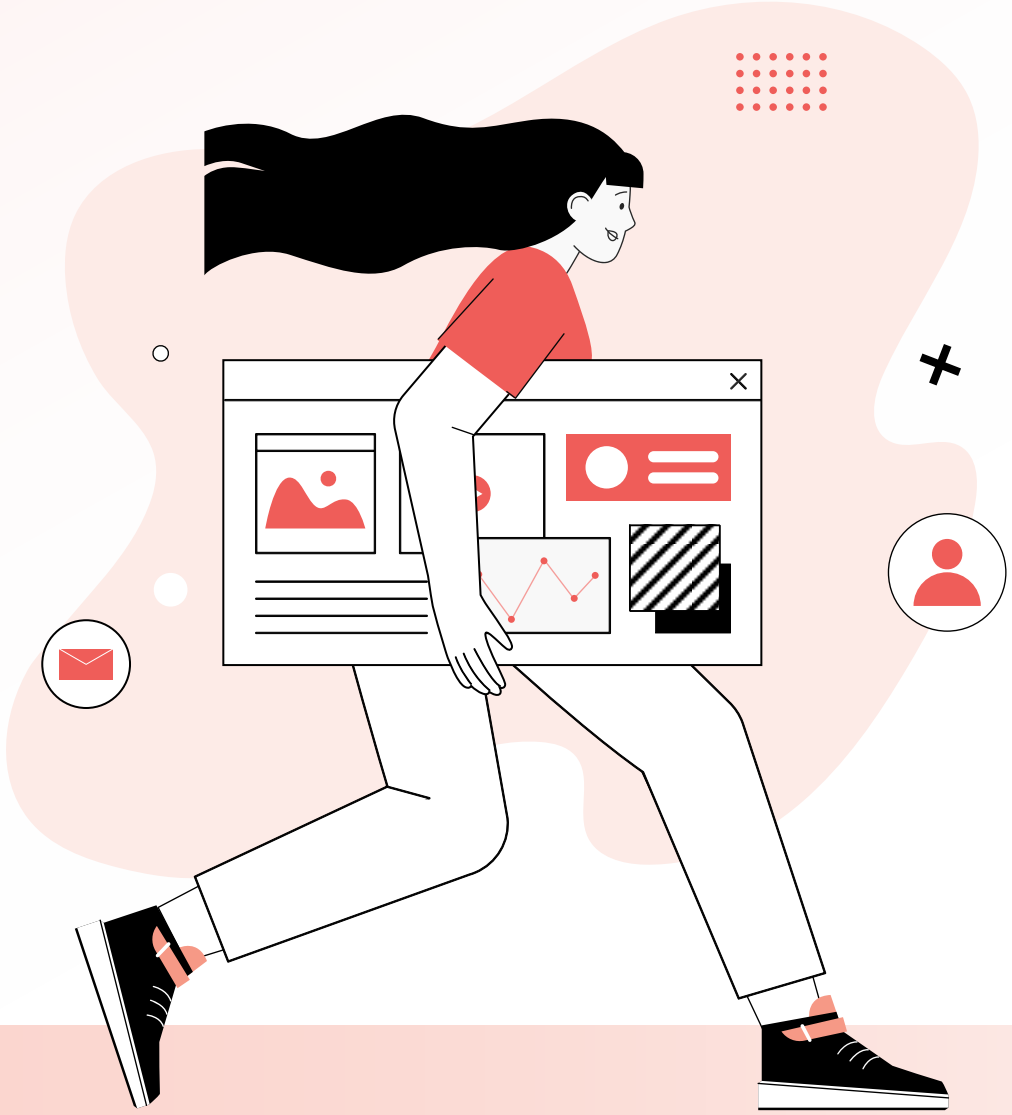
*Part 5

Cookies and Session

Firaol K. (Msc.)

firaol.kapita@gmail.com

https://t.me/firafiran

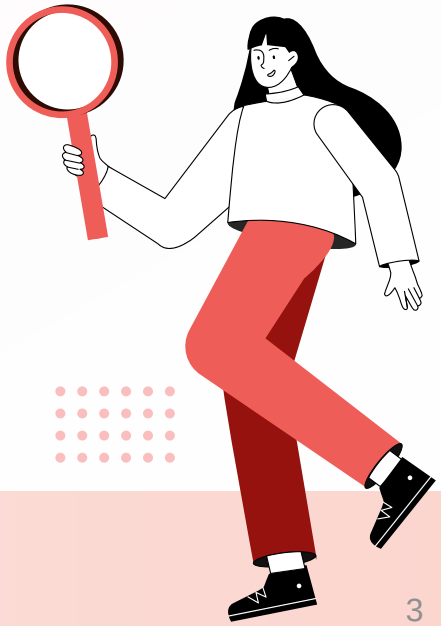


KEY POINT

- ✓Cookies
- ✓Create cookies
- ✓Sessions
- ✓Start session
- ✓Modify session
- ✓Access session
- ✓Destroy session

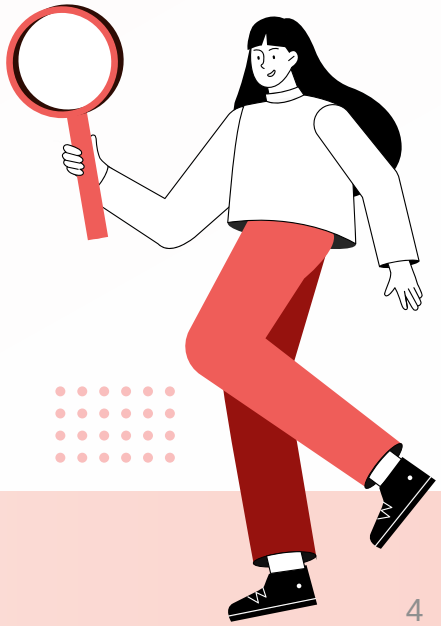
What is a Cookie?

- The worldwide web is powered by HTTP protocol, which is a stateless protocol.
- The mechanism of Cookies helps the server maintain the information of previous requests.
- PHP transparently supports HTTP cookies.



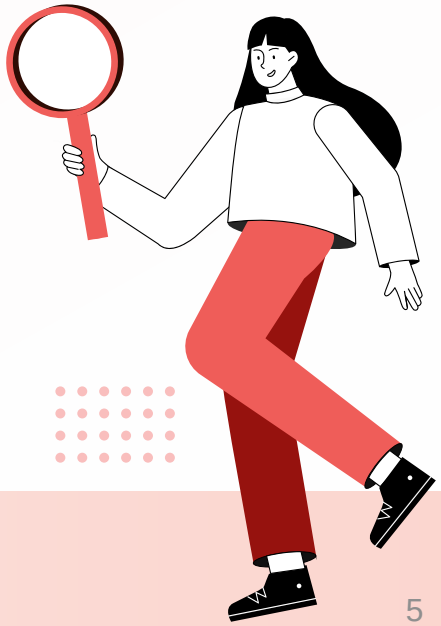
What is a Cookie?

- When building a website, we need to remember user information whether it's login details, preferences, or any other information.
- Two common ways to store this data are sessions and cookies

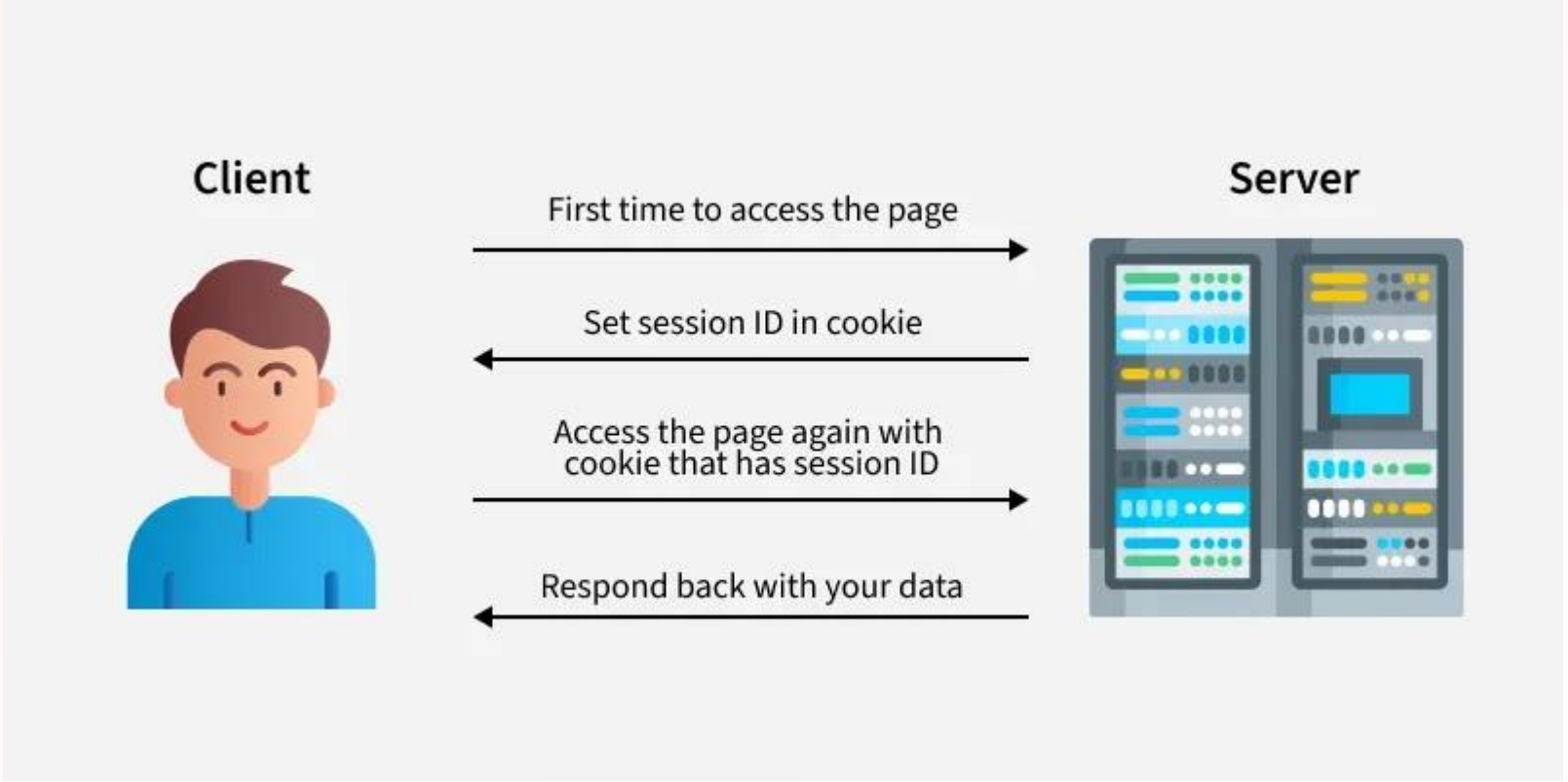


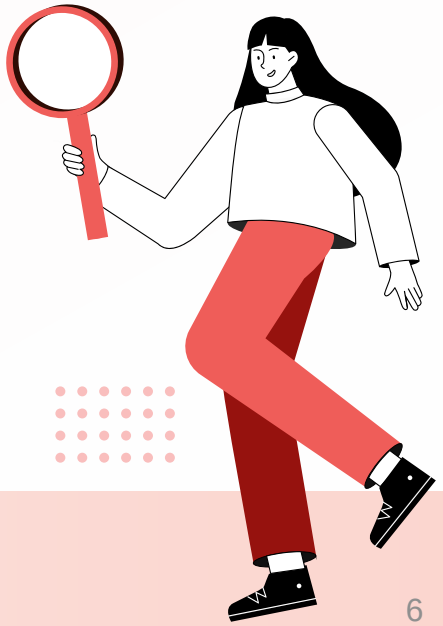
What is a Cookie?

- Cookies are small pieces of data stored in the user's browser.
- They help remember things like login status or preferences even after closing the website.
- Sessions store user data on the server, making them more secure and ideal for storing temporary or sensitive information.



What is a Cookie?

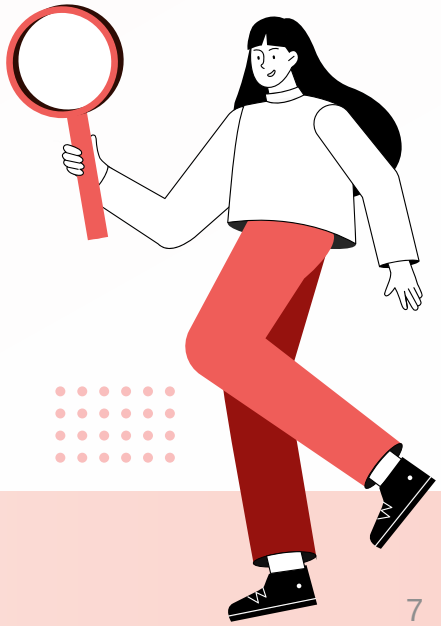




What is a Cookie?

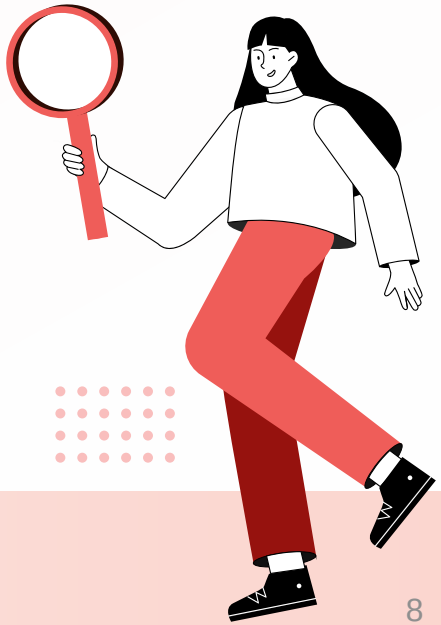
Cookies: Used for remembering login states and tracking users across multiple sessions.

Sessions: Used for temporary and secure storage during a single login session.



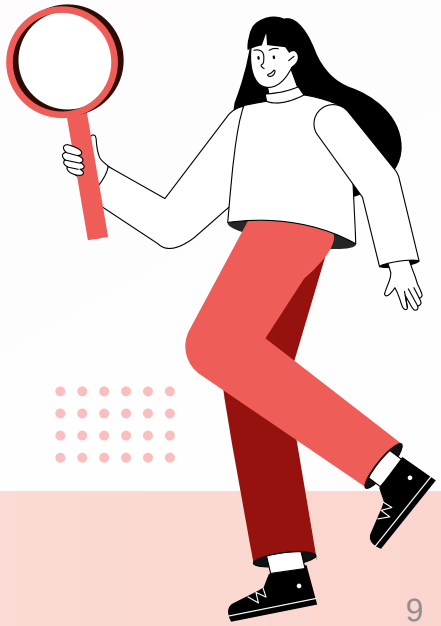
What is a Cookie?

- Cookies are small data stored on the client side (browser) as key-value pairs.
- They are commonly used for session management, user preferences, and behaviour tracking.
- When a user loads a website, the browser sends the stored cookies with the request, allowing the server to track and personalize the user's experience.



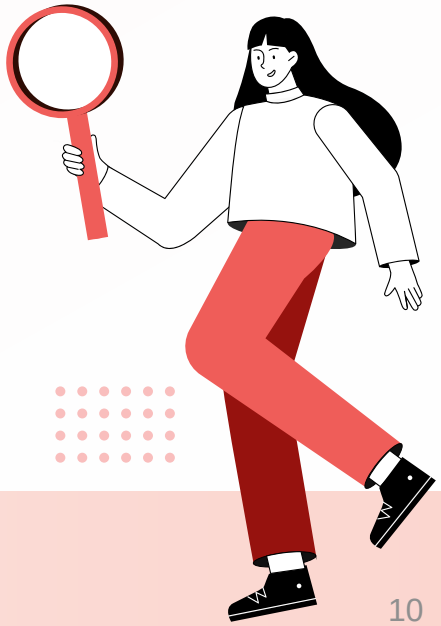
Features of Cookies

- **Client-Side Storage:** Data is stored in the user's browser, reducing the server's load.
- **Persistence:** Can be set to expire after a specific time, allowing data to persist across sessions.
- **Small Storage Size:** Limited to 4KB per cookie, making it suitable for lightweight data like preferences and authentication tokens.



Features of Cookies

- **Security:** Can be configured with HttpOnly, Secure (to allow only HTTPS), and SameSite (to prevent cross-site attacks).
- **Automatic Transmission:** Cookies are automatically sent with every request to the same domain, enabling tracking and authentication.
- **User Tracking:** Commonly used for session tracking, remembering login status, and personalization across different visits.





- A cookie is created with the **setcookie()** function.

Syntax

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

- Only the *name* parameter is required. All other parameters are optional.



PHP Create/Retrieve a Cookie

- The following example creates a cookie named "user" with the value "John".
- The cookie will expire after 30 days (86400 * 30). The "/" means that the cookie is available in entire website (otherwise, select the directory you prefer).
- We then retrieve the value of the cookie "user" (using the global variable $_COOKIE).
- We also use the **isset()** function to find out if the cookie is set:



```
1. <?php
2. $cookie_name = "user";
3. $cookie_value = "John";
4. setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1 day
5. ?>
```

HTML

```
1.  <html>
2.  <body>
3.  <?php
4.  if(!isset($_COOKIE[$cookie_name]))
5.  {
6.  echo "Cookie named '" . $cookie_name . "' is not set!";
7.  }
8.  else
9.  {
10. echo "Cookie '" . $cookie_name . "' is set!<br>";
11. echo "Value is: " . $_COOKIE[$cookie_name];
12. }
13. ?>
14. </body>
15. </html>
```





Check if Cookies are Enabled...

small script that checks whether cookies are enabled.

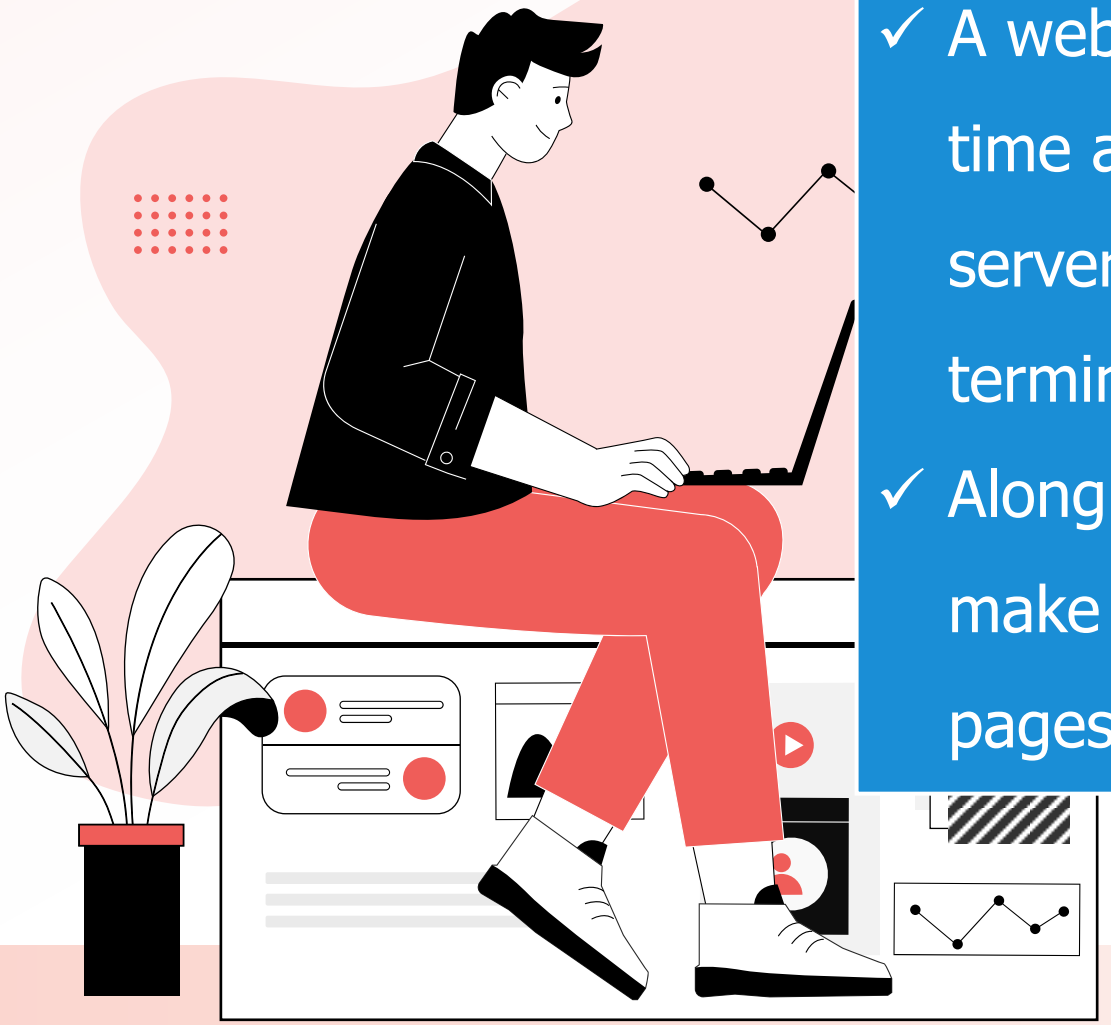
```
1.  <?php
2.  setcookie("test_cookie", "test", time() + 3600, '/');
3.  ?>
4.  <html>
5.  <body>
6.  <?php
7.  if(count($_COOKIE) > 0) {
8.   echo "Cookies are enabled.";
9.  }
10. Else
11.  {
12. echo "Cookies are disabled.";
13. }
14. ?>
15. </body>
16. </html>
```





PHP Sessions

PHP Sessions



- ✓ A web session is the time duration between the time a user establishes connection with a server and the time the connection is terminated.
- ✓ Along with the cookies, the session variables make the data accessible across the various pages of an entire website.

PHP Sessions



- ✓ A session is a way to store information (in variables) to be used across multiple pages.
- ✓ Unlike a cookie, the information is not stored on the users computer.
- ✓ A session is started with the session_start() function.
- ✓ Session variables are set with the PHP global variable: $_SESSION.

Key Characteristics of PHP Sessions:



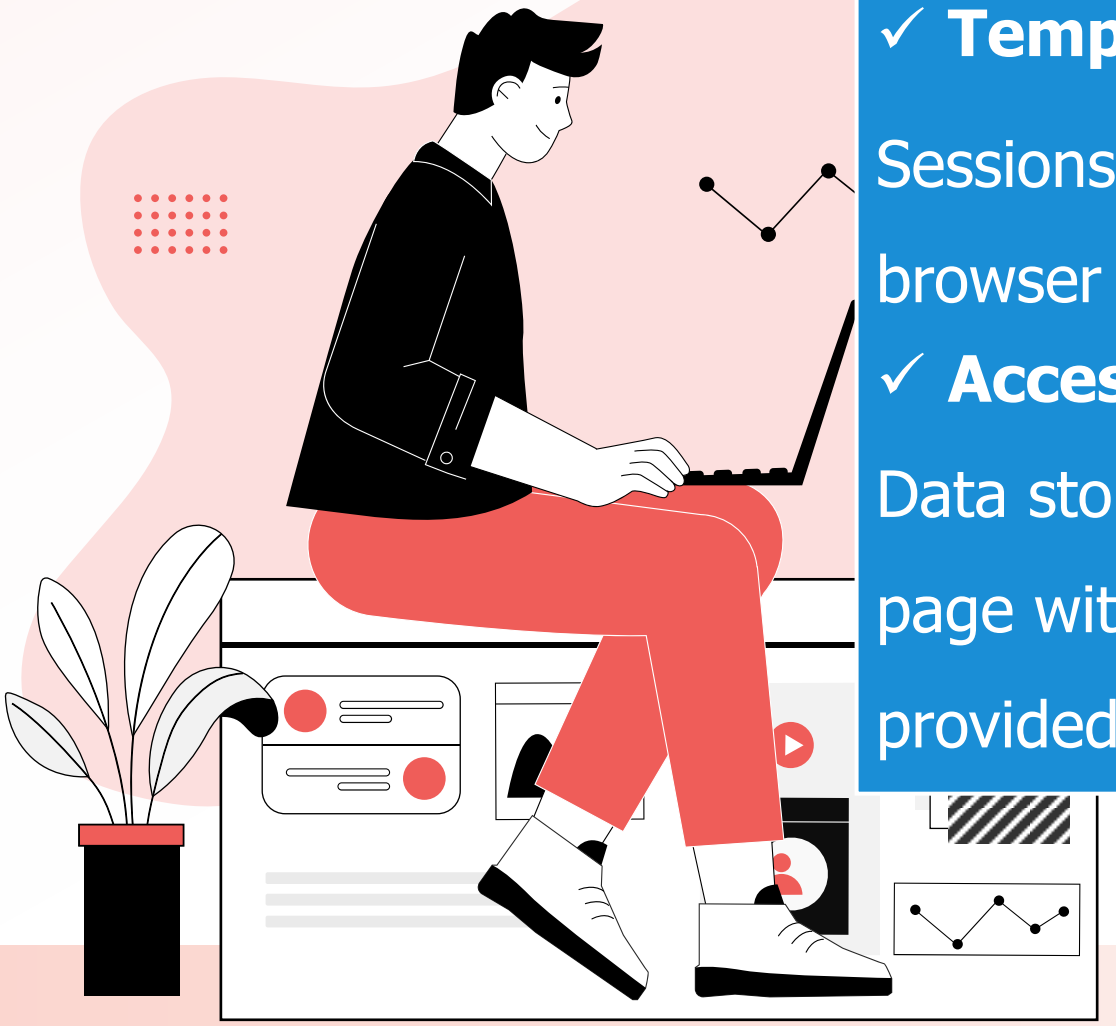
✓ **Server-Side Storage:**

Session data is stored on the server, not on the user's computer, enhancing security compared to cookies for sensitive information.

✓ **Session ID:**

Each user is identified by a unique session ID, typically stored in a cookie on the client-side, allowing the server to link requests to the correct session data.

Key Characteristics of PHP Sessions:



✓ **Temporary Storage:**

Sessions usually expire when the user closes their browser or after a configured period of inactivity.

✓ **Accessibility Across Pages:**

Data stored in $_SESSION is accessible on any page within the same session, provided session_start() is called.



Starting Sessions

- ✓ The session_start() function initiates a session
- ✓ This function must be called at the very **beginning** of any script that intends to use session variables.

Starting a Session



```
<?php


session_start();

// ... rest of your code


?>
```



Storing Session Data

- ✓ User-specific data is stored in the superglobal associative array $_SESSION.
- ✓ This array acts as a container for key-value pairs, where keys are descriptive names and values are the data to be stored.

Storing a Session



```
<?php

session_start();

$_SESSION["username"] = "john";

?>
```



Accessing Session Data

- ✓ On subsequent pages within the same session, the stored data can be accessed through the $_SESSION array after calling session_start().

Accessing a Session



```
<?php

session_start();

echo "Welcome back, " . $_SESSION["username"];

?>
```



Modifying Session Data

- ✓ Existing session variables can be updated by assigning new values to their respective keys within the $_SESSION array.

Modifying a Session



```
<?php

session_start();

$_SESSION["username"] = "james"; // Update username

?>
```



Destroying Session Data

- ✓ To end a session and clear all associated data, the session_destroy() function is used.
- ✓ This function removes the session data from the server.

Destroying a Session



```
 <?php

session_start();

session_destroy();

?>
```



!(Started)

Thank you!